Memory Management

The Need for Memory Management

- Memory is divided between OS and user
- In Multiprogramming, it must be subdivided to accommodate multiple processes
- Memory is cheap today, and getting cheaper
  - But applications are demanding more and more memory, there is never enough!
- Memory I/O is slow compared to a CPU
  - Sufficient memory must be allocated to ready processes
  - MM involves swapping blocks of data from secondary storage
  - OS must time swapping to maximise the CPU's efficiency

Memory Management

- Subdividing memory to accommodate multiple processes
- Memory needs to be allocated efficiently to pack as many ready processes into memory as possible to consume available processor time
- should be able to run a program whose size is larger than the available real memory

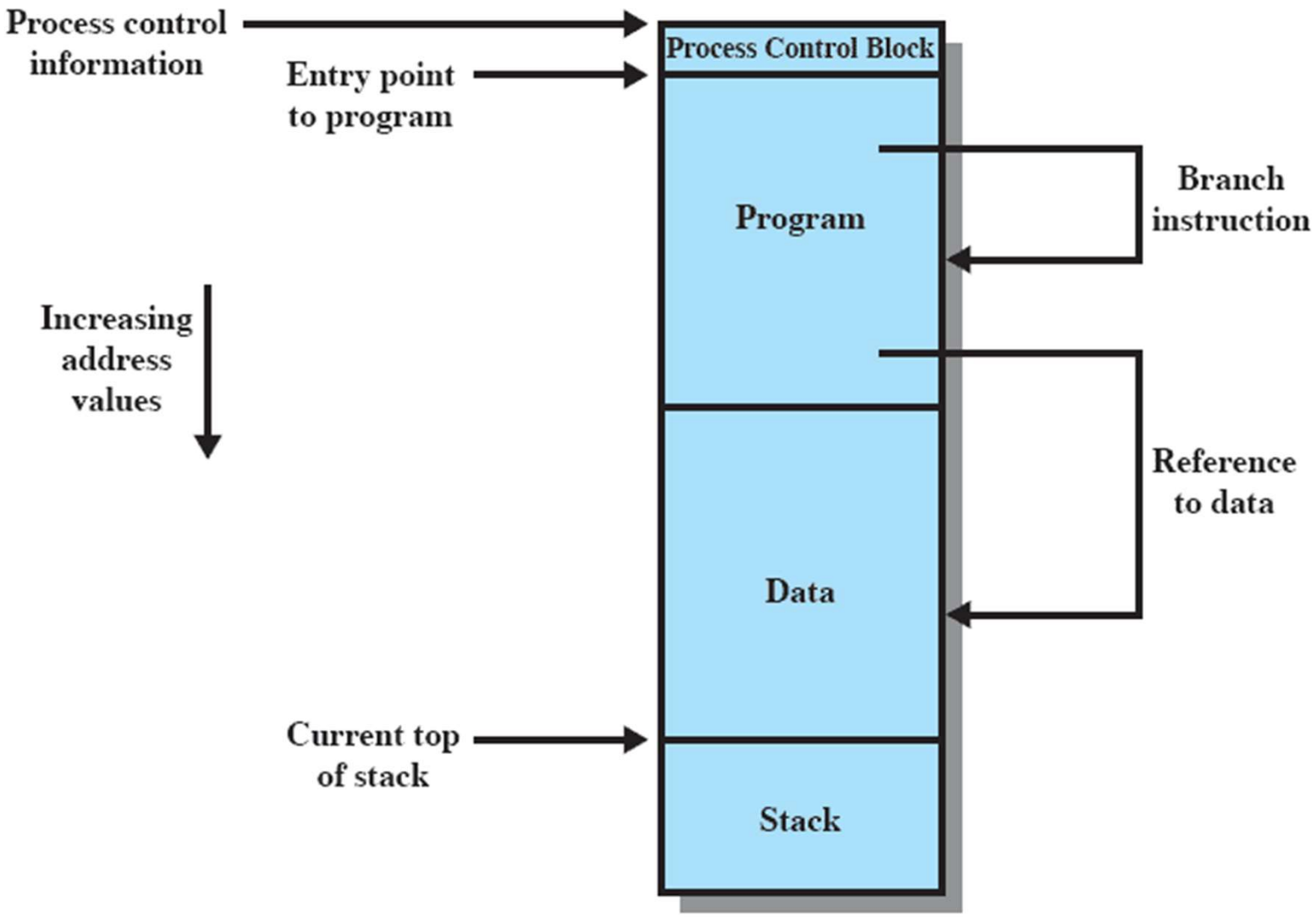
Memory Management Requirements

- Memory management is intended to satisfy the following requirements:
  - Relocation
  - Protection
  - Sharing
  - Logical organisation
  - Physical organisation

Memory Management Requirements

- **Relocation**
  - Programmer (or the compiler) does not know where the program will be placed in memory when it is executed
  - Memory references in the code must be translated to actual physical memory addresses
  - While the program is executing, it may be swapped to disk and returned to main memory at a different location (relocated)

Addressing Requirements for a Process



Memory Management Requirements (contd)

- **Protection**
  - Processes need to acquire permission to reference memory locations for reading or writing purposes
  - Location of a program in main memory is unpredictable
  - Memory references generated by a process must be checked at run time
    - OS cannot anticipate all of the memory references a program will make
- **Sharing**
  - Advantageous to allow each process access to the same copy of the program rather than have their own separate copy
  - Memory management must allow controlled access to shared areas of memory without compromising protection

Memory Management Requirements (contd)

- **Logical Organization**
  - Programs are written in modules, that can be shared
  - Modules are written and compiled independently
  - Different degrees of protection given to modules (read-only, execute-only)
  - OS and hardware should deal with user programs and data in forms of modules

Segmentation is one of the techniques that can realize this

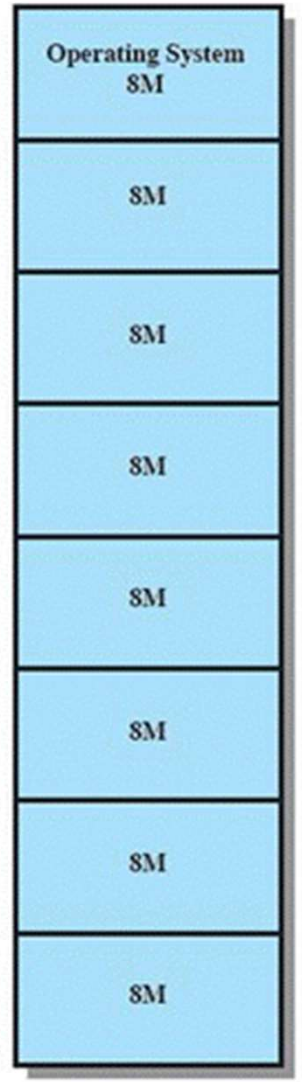
- **Physical Organization**
  - Memory is organized in two levels: main and secondary
  - moving information between the two levels is OS task

Memory Management Techniques

- Principle operation of MM is to bring processes in main memory. For that there are different techniques
  - Fixed Partitioning
  - Dynamic Partitioning
  - Simple Paging
  - Simple Segmentation
  - Virtual Memory Paging
  - Virtual Memory Segmentation

Fixed Partitioning

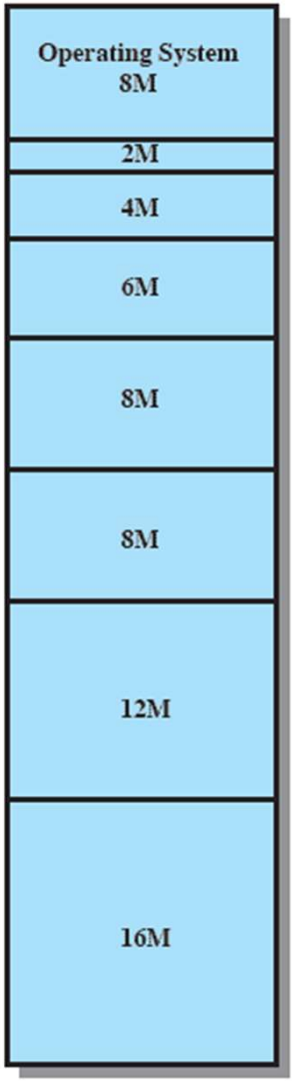
- Main memory divided into static partitions
  - Equal size partition
  - Unequal sized partition
- Equal-size partitions
  - Any process whose size ≤ the partition size can be loaded into any available partition
  - The OS can swap a process out of a partition, if a Ready process cannot find empty partition
  - Difficulties
    - A program may not fit in a partition: programmer must design the program with overlays
    - Maximum active processes fixed
    - Main memory use is inefficient: Small programs use entire partition. This is results in ***internal fragmentation*** (wasted space internal to a partition due to the fact that the block of data loaded is smaller than the partition)



(a) Equal-size partitions

Fixed Partitioning: Unequal-sized Partitions

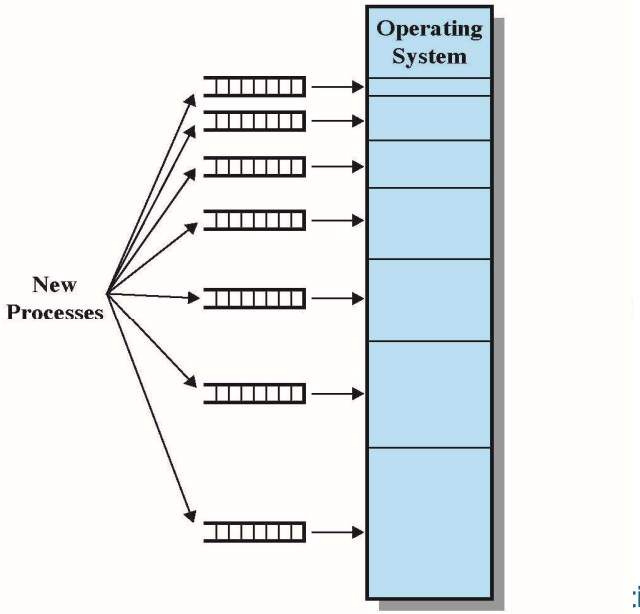
- Programs up to 16M can be loaded without overlay
- Smaller programs can be placed in smaller partitions, reducing internal fragmentation
- lessens the problem, but not solved
  - Modules greater than 16MB still need overlay
  - Internal fragmentation is still there
- Disadv
  - Limits number of active processes
  - Since partition size are preset at system generation, the layout might not fit the actual jobs resulting in large internal fragmentation and many overlays

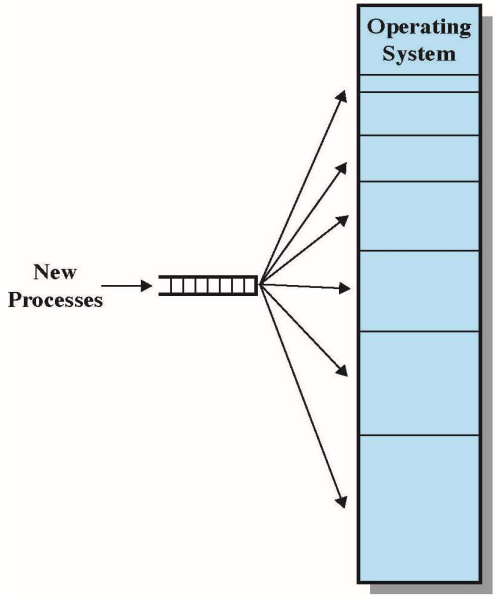


(b) Unequal-size partitions

Placement Algorithms with Partitioning

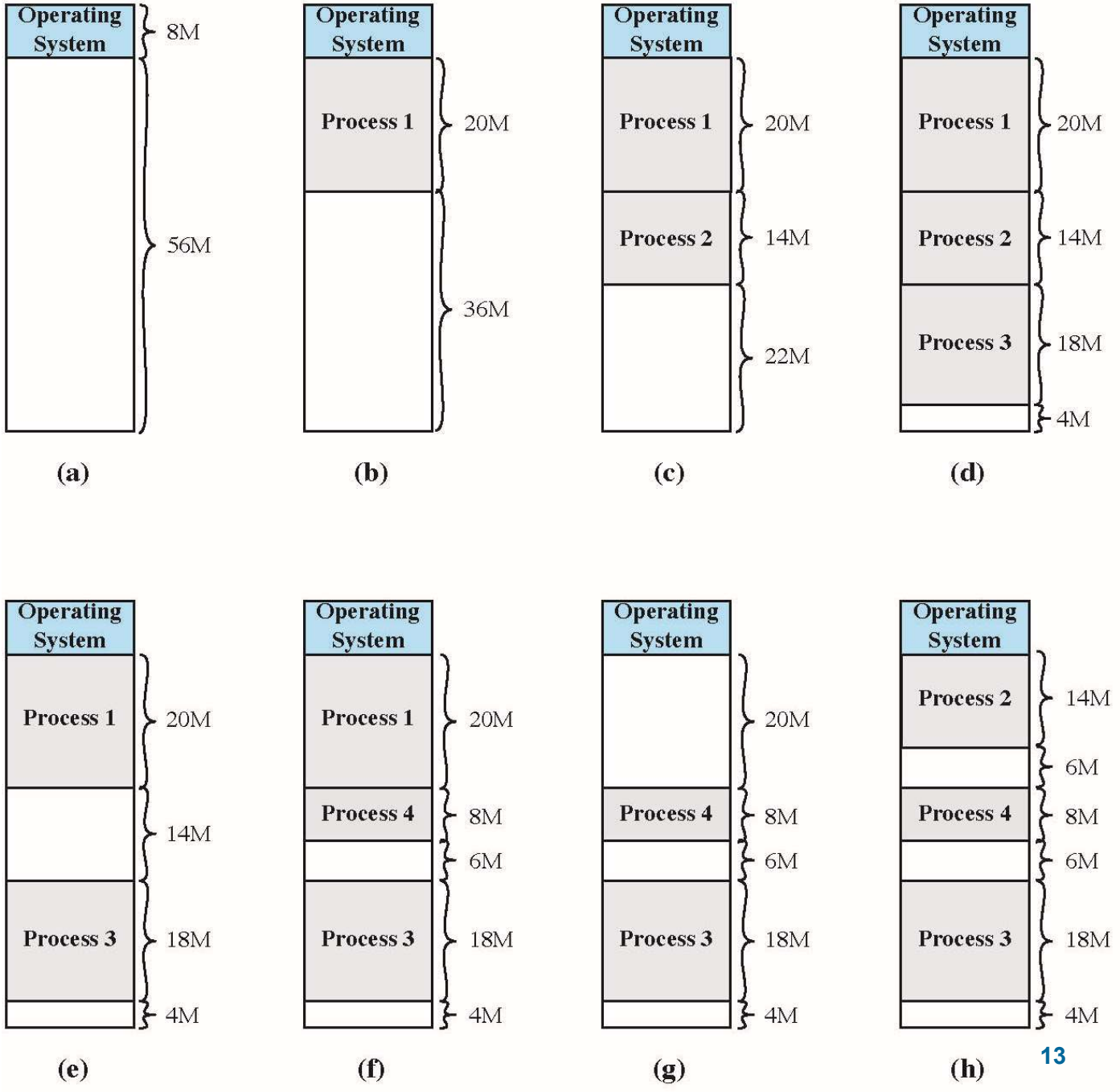
- **Equal-size partitions:** strategy is trivial: assign process to ANY partition
- **Unequal-size partitions**
  - assign each process to the *smallest partition* within which it will *fit*.
    - Scheduling queue is needed for each partition
    - Adv: minimize wasted memory within a partition: internal fragmentation
    - Disadv: large partition can remain unused, while small jobs queuing
  - Use a single queue for all partitions. When a new process arrives, select smallest available partition that fits.
    - Adv: flexible
    - Disadv: increase internal fragmentation





Dynamic Partitioning

- Partitions are of variable length and number
- Processes are allocated exactly as much memory as required
- Eventually holes will start appearing in the memory. This is called external fragmentation
- Must use compaction to shift processes so they are contiguous and all free memory is in one block -- overhead



Memory Fragmentation

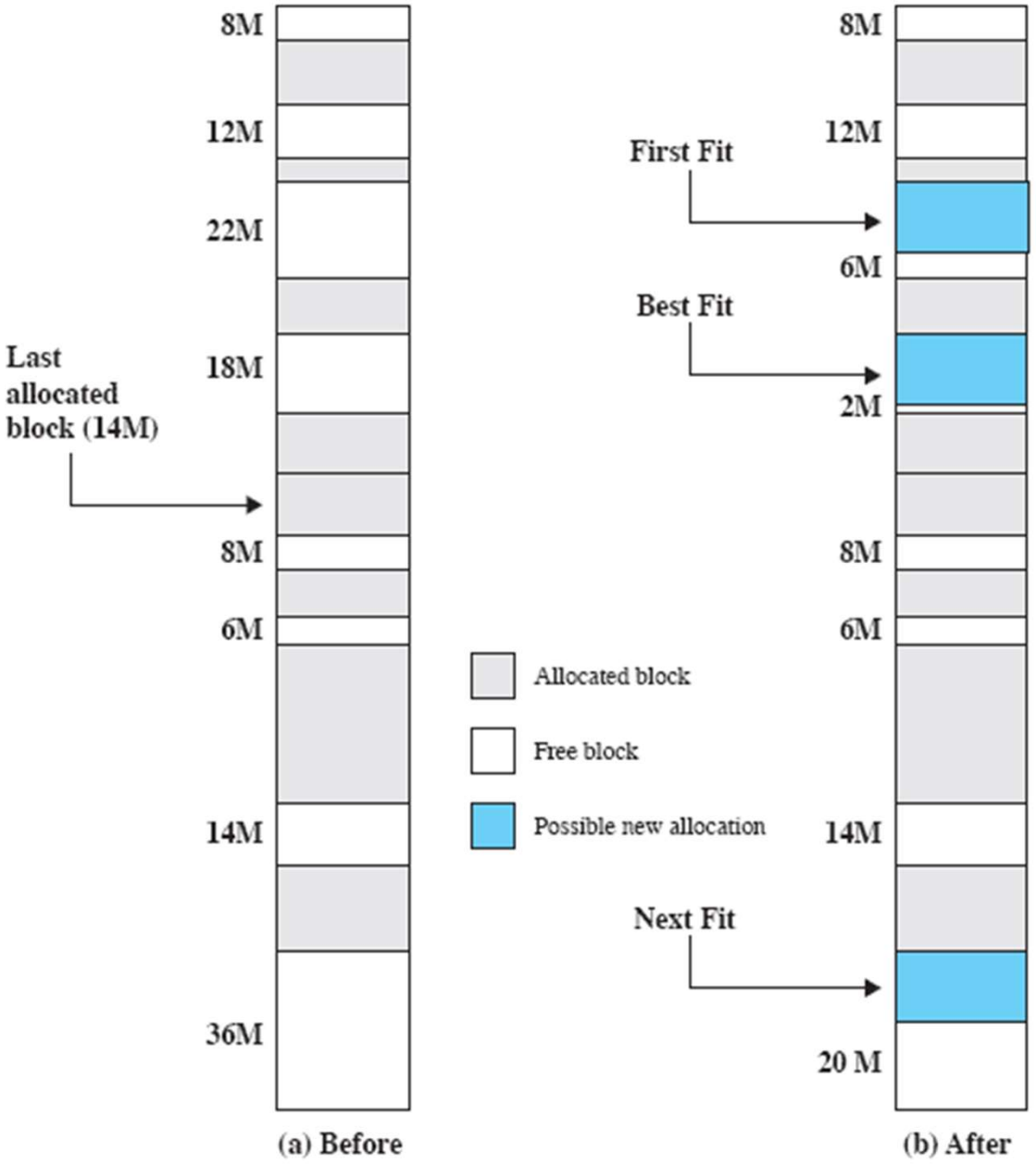
- As memory is allocated /deallocated fragmentation occurs
- External fragmentation
  - the memory that is external to all partitions becomes increasingly fragmented resulting in enough space existing to start a process, but it is not contiguous, so it cannot start
- Internal fragmentation
  - wasted space internal to a partition due to the fact that the block of data loaded is smaller than the partition
- Statistical analysis: given *N* allocated blocks, another 0.5 *N* blocks will be lost due to fragmentation
  - On average, 1/3 of memory is unusable (50-percent rule)
- Solution – Compaction.
  - Move allocated memory blocks so they are contiguous
  - Run compaction algorithm periodically
    - How often?
    - When to schedule?

Allocation Strategies with Dynamic Partitioning

- Operating system must decide which free block to allocate to a process
- **First Fit**
  - Allocate the first spot in memory that is big enough to satisfy the requirements.
- **Best Fit**
  - Chooses the block that is closest in size to the request
- **Next Fit**
  - Scan memory from the location of the last placement and choose the next available block that is large enough.

Allocation of 16-MB block

- For allocation request of 16 MB
- **Best-fit** results in a 2-Mbyte fragment
- **First-fit** results in a 6-Mbyte fragment
- **Next-fit** results in a 20-Mbyte fragment



Which Allocation Strategy?

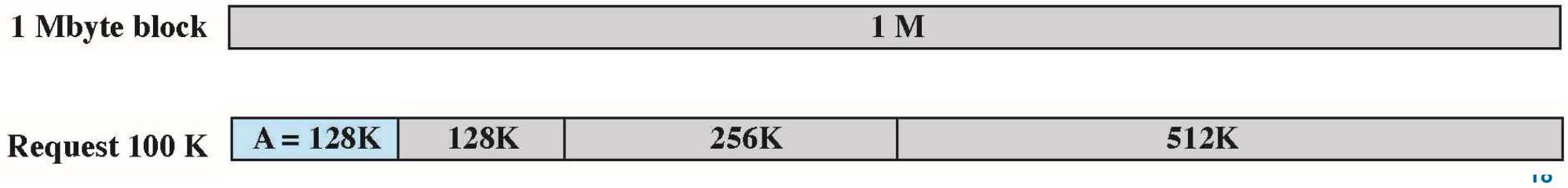
- **First-fit** is not only the *simplest* but usually the *best* and the *fastest* as well.
  - May have many processes loaded in the front end of memory that must be searched over when trying to find a free block
- **Next-fit** will more frequently lead to an allocation from a free block at the end of memory.
  - Results in fragmenting the largest block of free memory.
  - Compaction may be required more frequently.
- **Best-fit** is usually the worst performer!
  - Since smallest block is found for process, the smallest amount of fragmentation is left. Memory is littered by too many small blocks
  - Memory compaction must be done more often

Buddy System

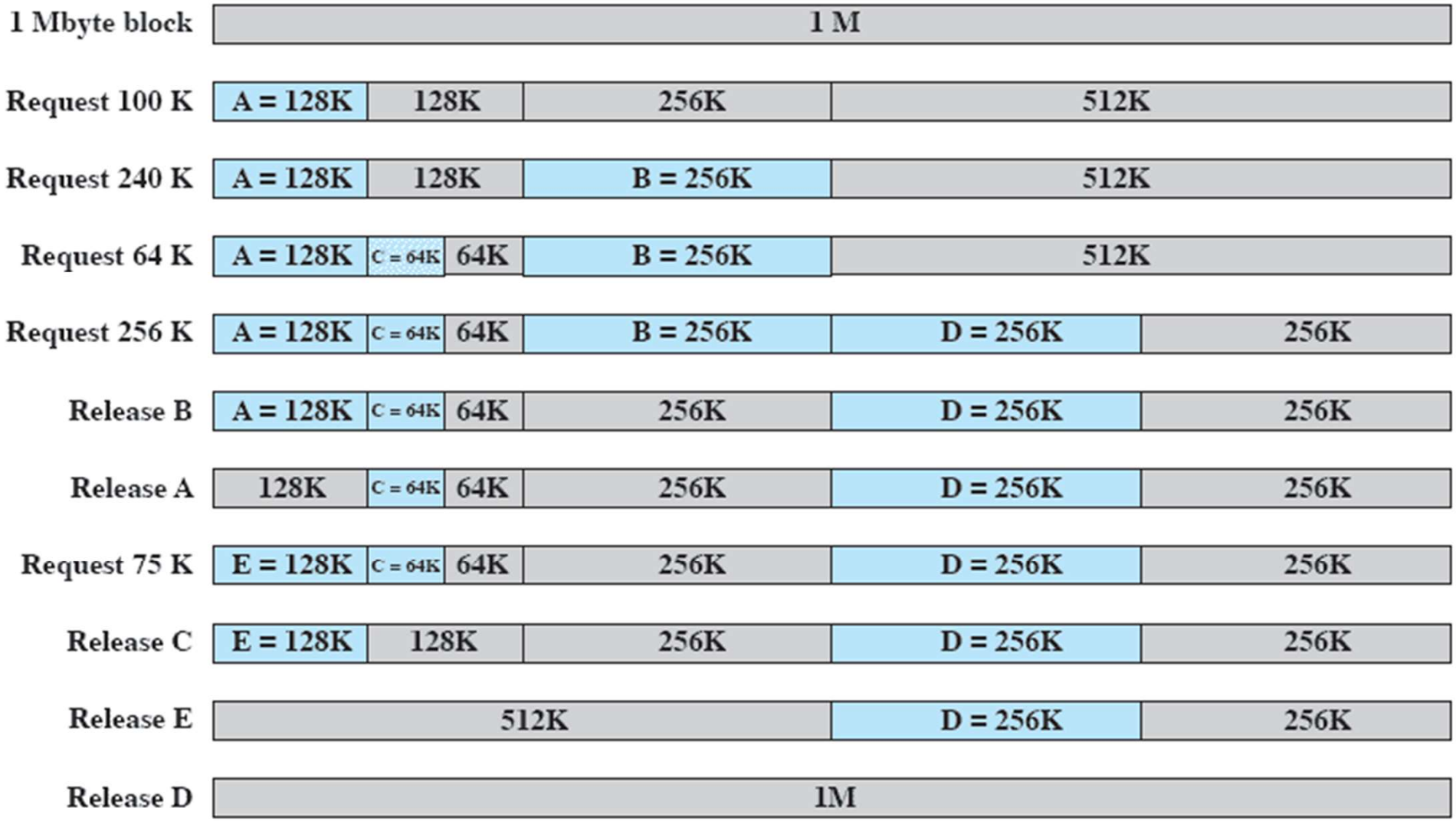
- Buddy System is a compromise between fixed and dynamic partitioning
- memory blocks are available of size 2^K *words,* $L \leq K \leq U$, where

2^L *smallest size block and* 2^U *is largest size block*

- Initially, entire space available is treated as a single block of 2^U
- If a request of size **s** is such that $\mathbf{2^{U-1} < s \leq 2^U}$, entire block of $\mathbf{2^U}$ is allocated
  - Otherwise block is split into two equal buddies
  - Splitting continues until smallest block greater than or equal to **s** is generated

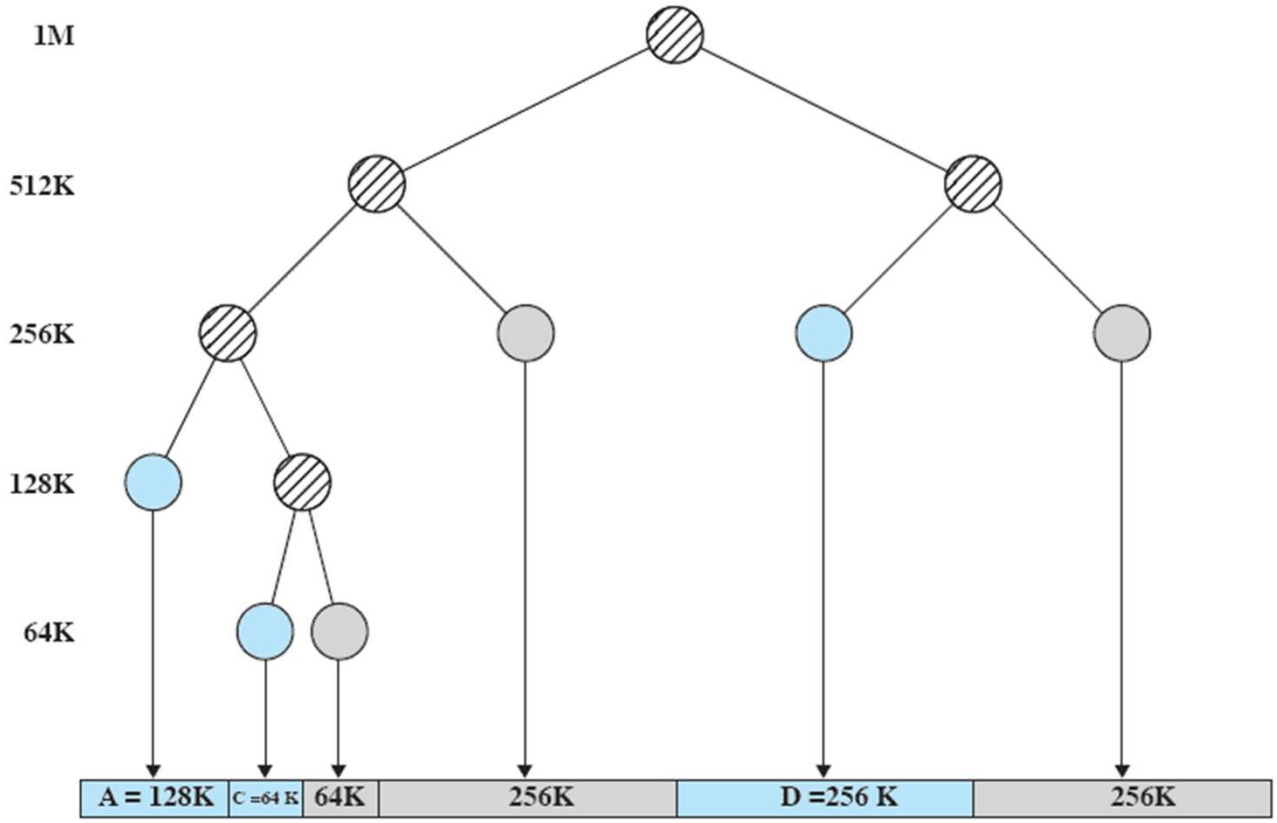


Example of Buddy System



Tree Representation of Buddy System

- The leaf nodes represent the current partitioning the memory.
- If two buddies are leaf nodes, then at least one must be allocated; otherwise they would be coalesced into a larger block.



After the Release B request

Addresses

- Logical
  - Reference to a memory location independent of the current assignment of data to memory.
- Relative
  - Address expressed as a location relative to some known point
- Physical or Absolute
  - The absolute address or actual location in main memory
- A translation must be made from both Logical and Relative addresses to Absolute address

Relocation

- When program loaded into memory the actual (absolute) memory locations are determined
- A process may occupy different partitions which means different absolute memory locations during execution, due to
  - Swapping
  - Compaction

Base/Bounds Relocation

- Base Register
  - Holds beginning physical address
  - Add to all program (relative) addresses
- Bounds Register
  - Used to detect accesses beyond the end of the allocated memory
  - May have length instead of end address
  - Provides protection to system
- Easy to move programs in memory
  - These values are set when the process is loaded and when the process is swapped in

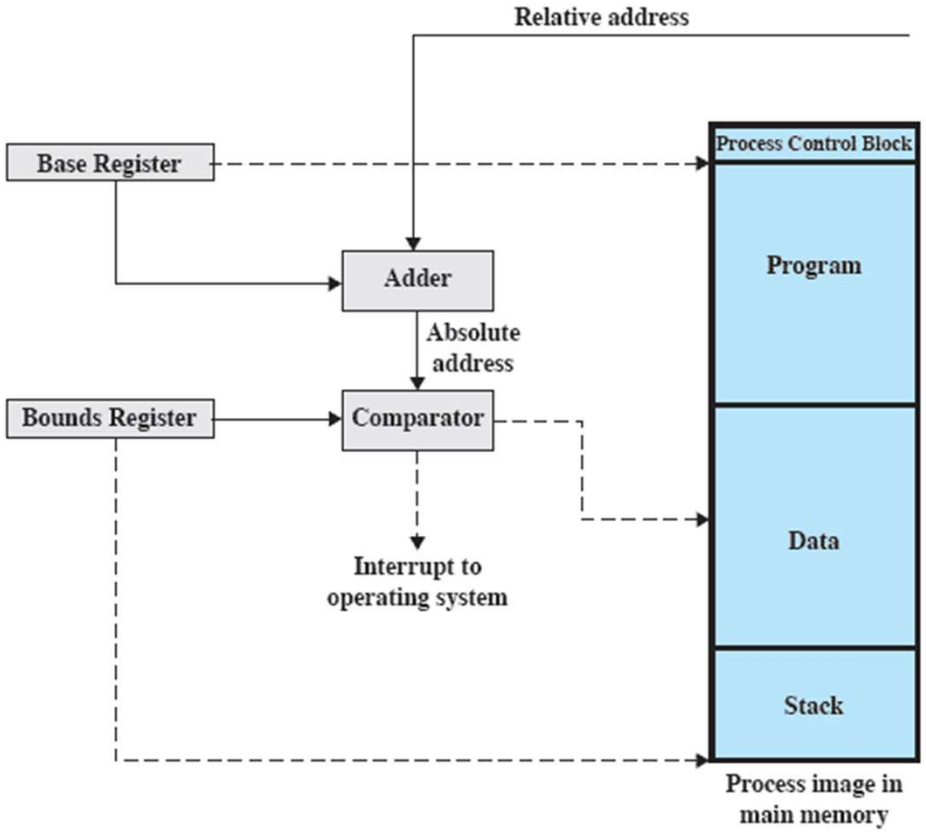
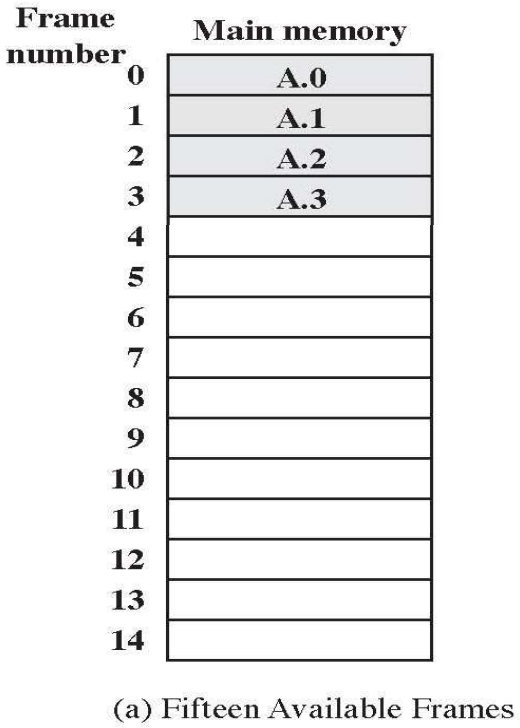


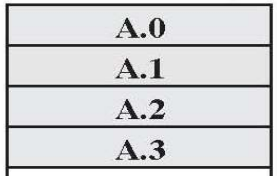
Figure 7.8 Hardware Support for Relocation

Paging

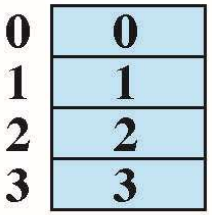
- Partition memory into small equal fixed-size chunks and divide each process into the same size chunks
- The chunks of memory are called ***frames***
- The chunks of a process are called ***pages***
- Operating system maintains a page table for each process
  - Contains the frame location for each page in the process
  - Memory address consist of a page number and offset within the page



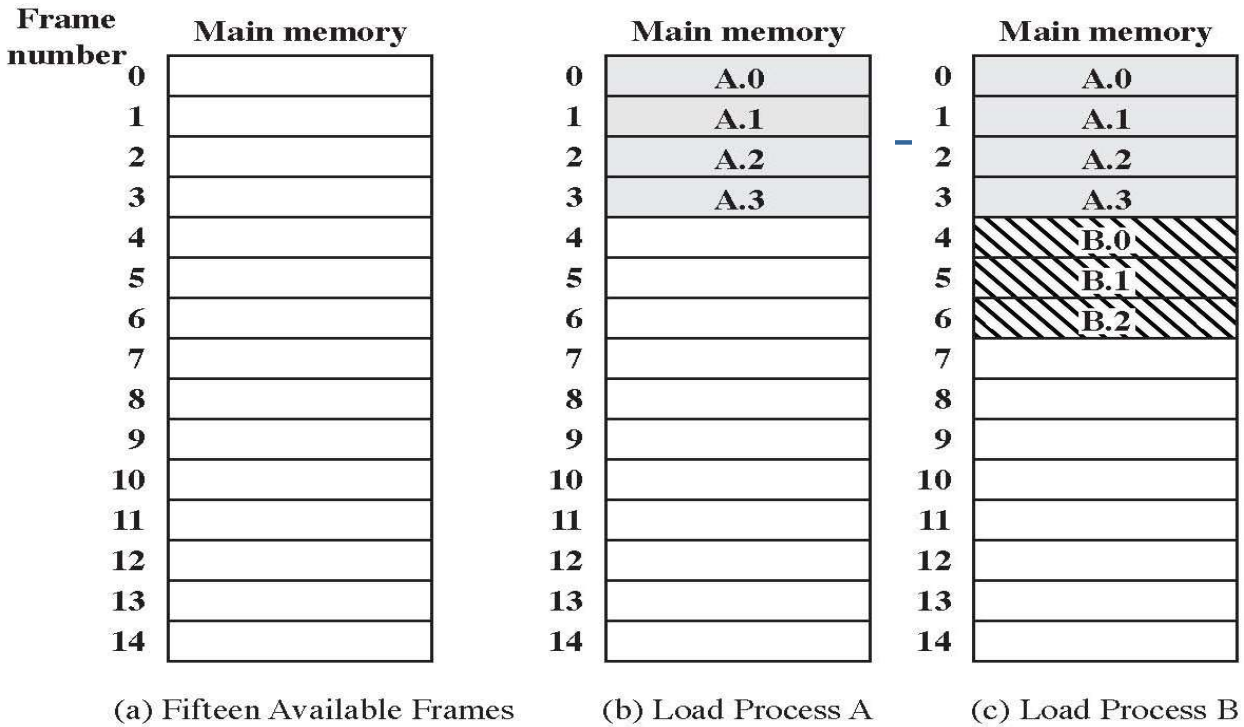
(a) Fifteen Available Frames

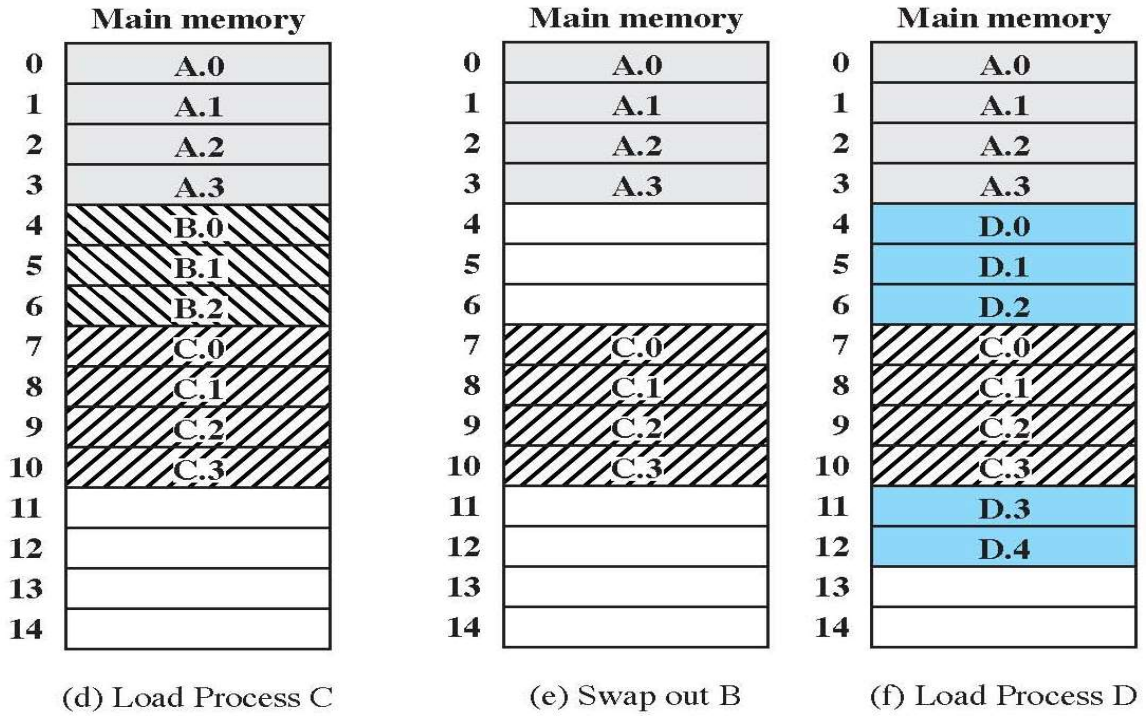


Pages of process A

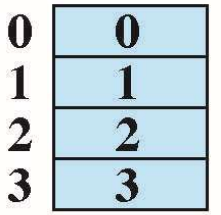


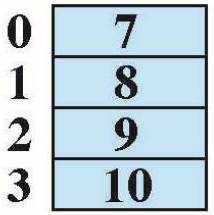
Process A page table

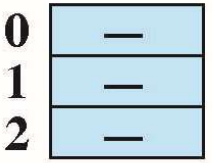

Frame number
Main memory
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
A.0
A.1
A.2
A.3
B.0
B.1
B.2
(a) Fifteen Available Frames
(b) Load Process A
(c) Load Process B

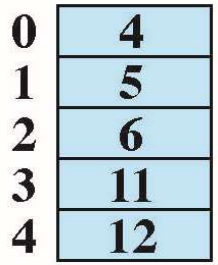

Main memory
A.0
A.1
A.2
A.3
B.0
B.1
B.2
C.0
C.1
C.2
C.3
D.0
D.1
D.2
D.3
D.4
(d) Load Process C
(e) Swap out B
(f) Load Process D

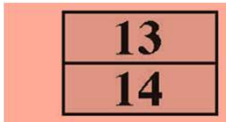
Page Tables


0 0
1 1
2 2
3 3
Process A page table


0 7
1 8
2 9
3 10
Process C page table


0 —
1 —
2 —
Process B page table


0 4
1 5
2 6
3 11
4 12
Process D page table


13
14
Free frame list

Paging

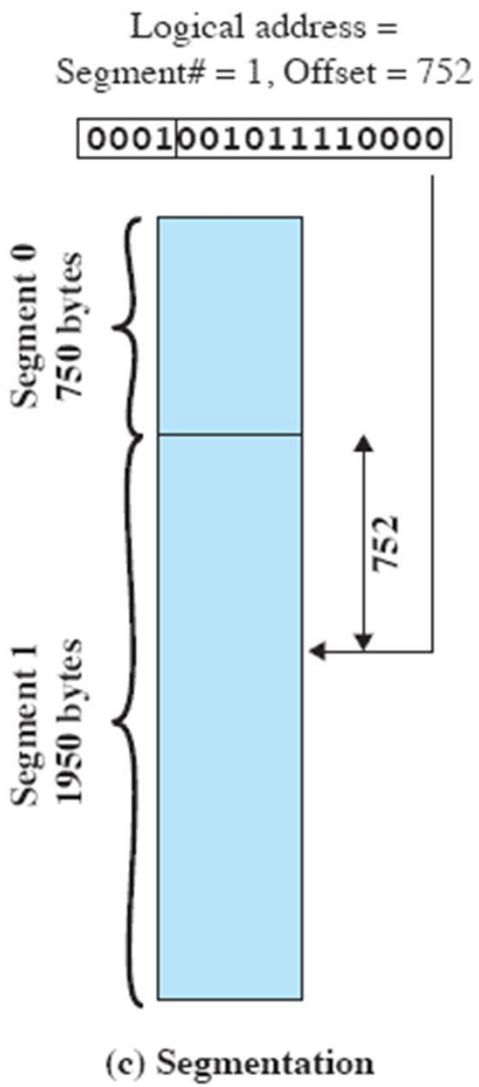
- Paging is similar to fixed partitioning, but with paging
  - the partitions are rather small
  - A program may occupy more than one partition;
  - and these partitions need not be contiguous
  - Internal fragmentation is limited only to a fraction of the last page of a process
  - There is no external fragmentation

Paging

- OS maintains
  - a single **free-frame list** of all frames in main memory
  - A **Page table** for each process
    - Contains an entry for each page of the process to be easily indexed by page number
    - Each page table entry contains the number of frame in main memory
- Addressing: hardware support is needed
  - Logical address (page number, offset) must be translated to physical address (frame number, offset).
  - the logical-to-physical address translation is done by processor hardware
  - Processor uses the page table to produce the physical address

Segmentation

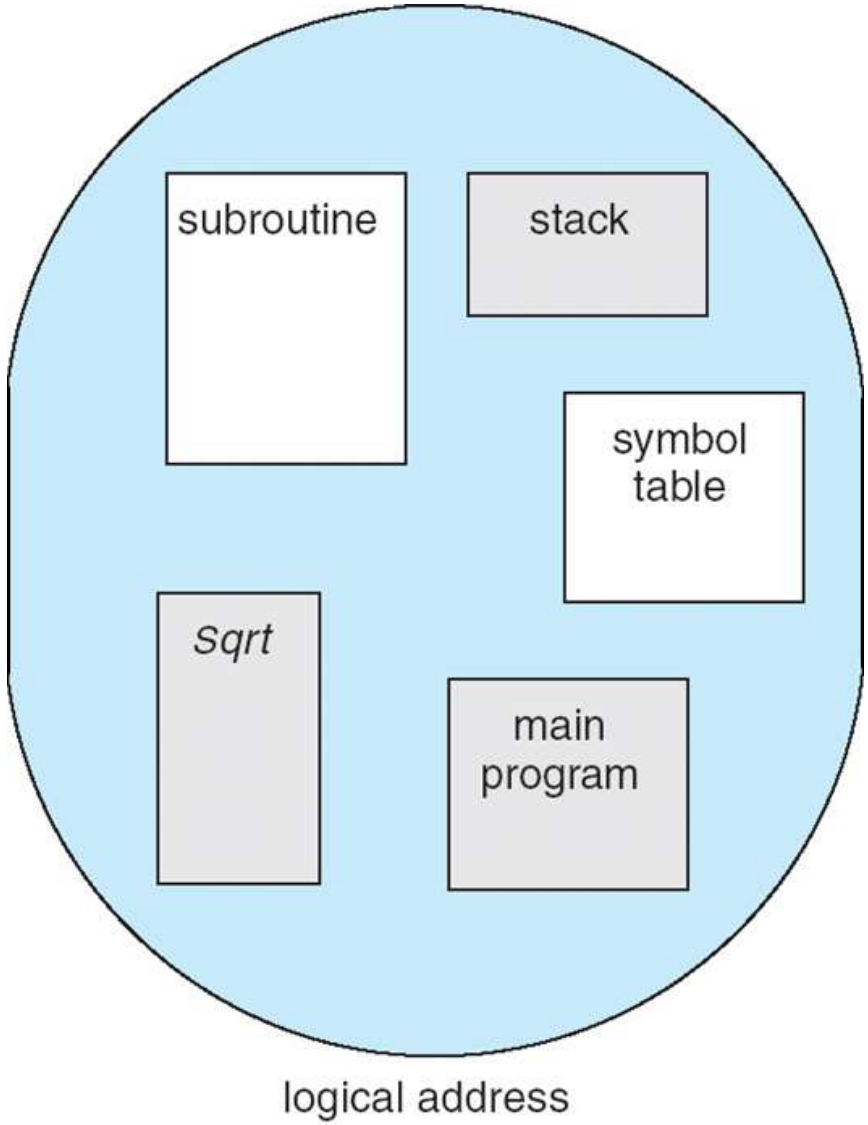
- A program can be subdivided into segments
  - Segments may vary in length
  - There is a maximum segment length
- Addressing consist of two parts
  - a segment number and an offset
- Paging is invisible to programmers, but segmentation is visible
  - Modules may be broken to multiple segments
  - Programmer or compiler will assign program and data to different segments
  - Programmer must be at least aware of maximum segment size limitation



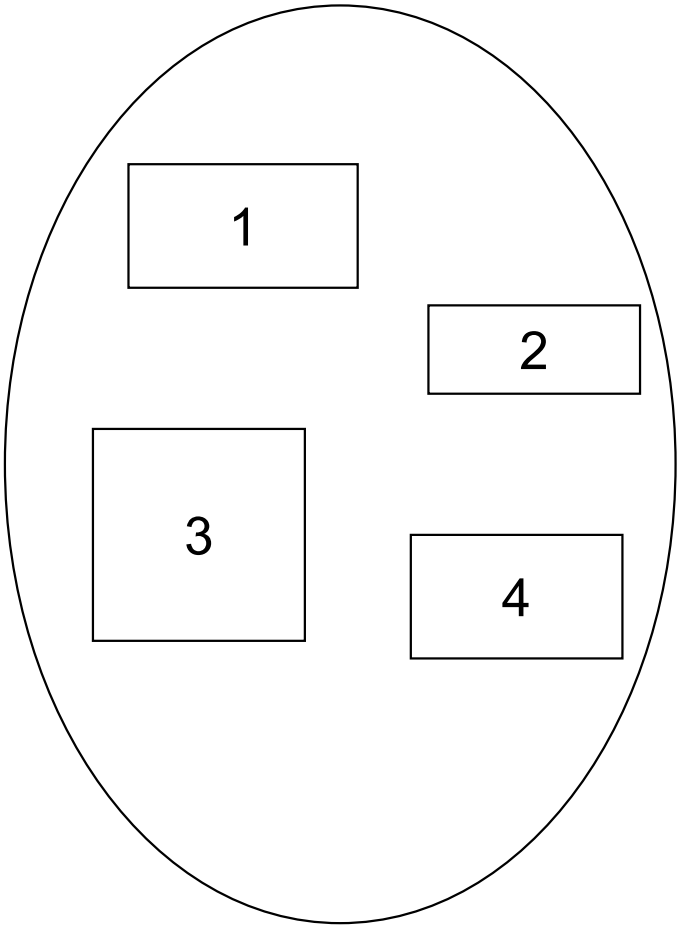
(c) Segmentation

Segmentation

- Memory-management scheme that supports user view of memory
- A program is a collection of segments
  - A segment is a logical unit such as:
    - main program
    - procedure
    - function
    - local variables, global variables
    - stack
    - symbol table
    - arrays



Logical View of Segmentation



user space

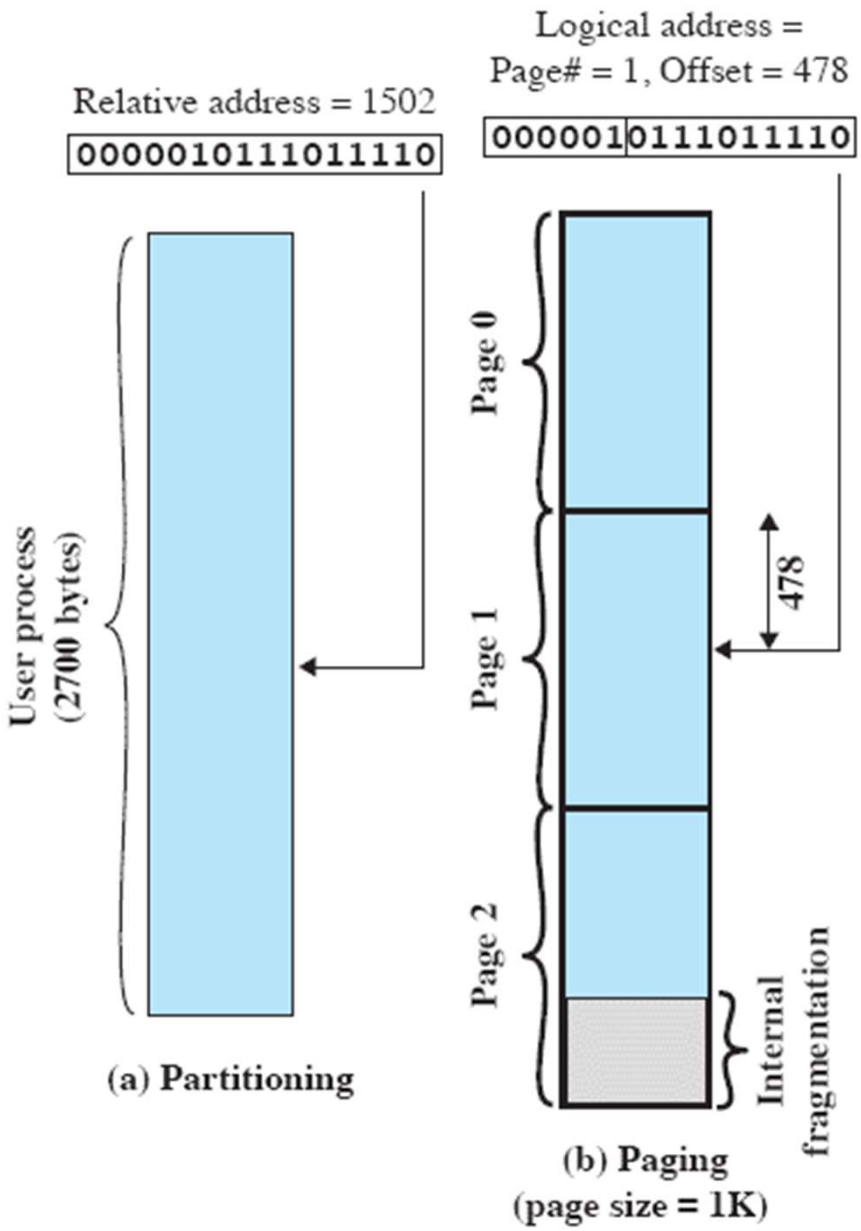
physical memory space

Segmentation

- Segmentation is similar to dynamic partitioning, but
  - A program may occupy more than one partition, and these partitions need not be contiguous
  - Segmentation still suffers from external fragmentation. However, because a process consists of a number of smaller pieces, the external fragmentation should be less
- Due to unequal-size segments, there is no simple relationship between logical addresses and physical addresses.
- Like paging, a segmentation scheme make use of
  - a segment table for each process and
  - a list of free blocks of main memory.
  - Each segment table entry would have to give
    - the starting address in main memory of the corresponding segment.
    - the length of the segment
- When a process enters the Running state, the address of its segment table is loaded into a special register used by the memory management hardware.

Logical Addresses

-Assume Page/Frame size is power of 2
- then relative address (defined with reference to the origin of the program)
and the logical address (expressed as a page number and offset) are the same
-Ex: assume16-bit addresses &page size=1K
- offset 1K=1024 = 2^{10} needs 10 bits
- 6 bits remain for page #, so a program consists of max 2^6 = 64 pages
- relative address 1502 corresponds to
  - offset of 478 (0111011110) on page 1 (000001)
  - which yields the same 16-bit number,
    0000010111011110
- ++ Using a page size of power of 2
  - logical address scheme is transparent to programmer, linker and assembler since logical address = relative address
  - easier to implement a function in hardware to perform dynamic address translation at run time



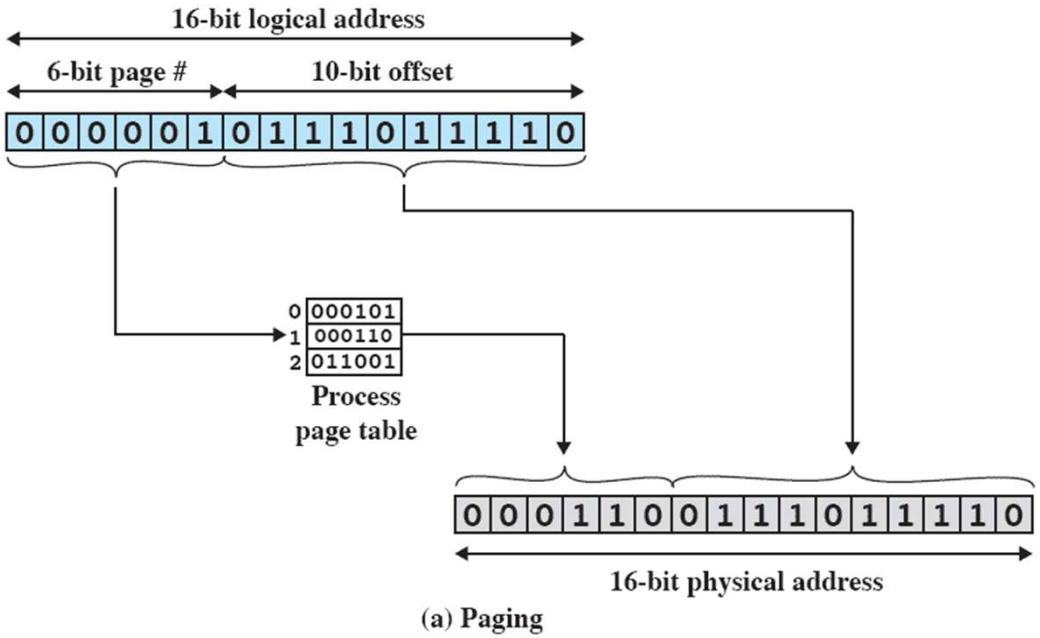
Paging

Consider an address of *n +m bits,*
- the leftmost n bits are the page #
- the rightmost m bits are the offset
- Here n =6 and m =10
- steps for address translation:

• Extract the page # as the leftmost *n* bits of the logical address.

• Use the page # as an index into the process page table to find the frame number, *k.*

• The starting physical address of the frame is $k \times 2^m$
- the physical address= $k \times 2^m$ + offset. - This physical address need not be calculated;: just append the frame number to the offset.



(a) Paging

Segmentation

Consider an address of *n +m bits,*
- the leftmost n bits are the segment #
- the rightmost m bits are the offset
- Here n =4 and m =12
- steps for address translation:

• Extract the segment # as the leftmost *n* bits of the logical address

• Use the segment # as an index into the process segment table to find starting physical address

• Compare the offset, rightmost m bits, to the length of the segment. If the offset ≥ length, the address is invalid.

• The desired physical address is the sum of the starting physical address of the segment plus the offset

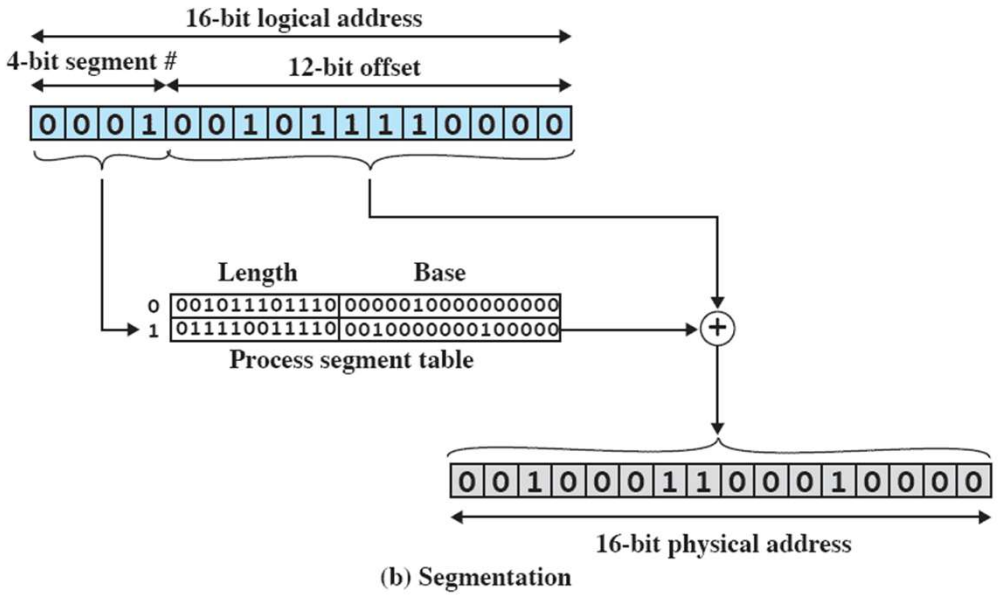


Figure 7.12 Examples of Logical-to-Physical Address Translation

Memory Management Techniques

Technique	Description	Strengths	Weaknesses
Fixed Partitioning	Main memory is divided into a number of static partitions at system generation time. A process may be loaded into a partition of equal or greater size.	Simple to implement; little operating system overhead.	Inefficient use of memory due to internal fragmentation; maximum number of active processes is fixed.
Dynamic Partitioning	Partitions are created dynamically, so that each process is loaded into a partition of exactly the same size as that process.	No internal fragmentation; more efficient use of main memory.	Inefficient use of processor due to the need for compaction to counter external fragmentation.
Simple Paging	Main memory is divided into a number of equal-size frames. Each process is divided into a number of equal-size pages of the same length as frames. A process is loaded by loading all of its pages into available, not necessarily contiguous, frames.	No external fragmentation.	A small amount of internal fragmentation.
Simple Segmentation	Each process is divided into a number of segments. A process is loaded by loading all of its segments into dynamic partitions that need not be contiguous.	No internal fragmentation; improved memory utilization and reduced overhead compared to dynamic partitioning.	External fragmentation.
Virtual Memory Paging	As with simple paging, except that it is not necessary to load all of the pages of a process. Nonresident pages that are needed are brought in later automatically.	No external fragmentation; higher degree of multiprogramming; large virtual address space.	Overhead of complex memory management.
Virtual Memory Segmentation	As with simple segmentation, except that it is not necessary to load all of the segments of a process. Nonresident segments that are needed are brought in later automatically.	No internal fragmentation, higher degree of multiprogramming; large virtual address space; protection and sharing support.	Overhead of complex memory management.

Memory Management Techniques

- **Fixed Partitioning**
  - Divide memory into partitions at boot time, partition sizes may be equal or unequal but don't change
  - Simple but has internal fragmentation
- **Dynamic Partitioning**
  - Create partitions as programs loaded
  - Avoids internal fragmentation, but must deal with external fragmentation
- **Simple Paging**
  - Divide memory into equal-size pages, load program into available pages
  - No external fragmentation, small amount of internal fragmentation

Memory Management Techniques

- **Simple Segmentation**
  - Divide program into segments
  - No internal fragmentation, some external fragmentation
- **Virtual-Memory Paging**
  - Paging, but not all pages need to be in memory at one time
  - Allows large virtual memory space
  - More multiprogramming, overhead
- **Virtual Memory Segmentation**
  - Like simple segmentation, but not all segments need to be in memory at one time
  - Easy to share modules
  - More multiprogramming, overhead